as it's safe to reclaim your list. You can then just call `delete_nodes` to iterate down the list and delete them ❹.

If the count is *not* zero after the decrement, it's not safe to reclaim the nodes, so if there are any ❺, you must chain them back onto the list of nodes pending deletion ❻. This can happen if there are multiple threads accessing the data structure concurrently. Other threads might have called `pop()` in between the first test of `threads_in_pop` ❶ and the "claiming" of the list ❷, potentially adding new nodes to the list that are still being accessed by one or more of those other threads. In figure 7.1, thread C adds node Y to the `to_be_deleted` list, even though thread B is still referencing it as `old_head`, and will thus try and read its `next` pointer. Thread A can't therefore delete the nodes without potentially causing undefined behavior for thread B.

To chain the nodes that are pending deletion onto the pending list, you reuse the `next` pointer from the nodes to link them together. In the case of relinking an existing chain back onto the list, you traverse the chain to find the end ❾, replace the `next` pointer from the last node with the current `to_be_deleted` pointer ❿, and store the first node in the chain as the new `to_be_deleted` pointer ⓫. You have to use `compare_exchange_weak` in a loop here in order to ensure that you don't leak any nodes that have been added by another thread. This has the benefit of updating the `next` pointer from the end of the chain if it has been changed. Adding a single node onto the list is a special case where the first node in the chain to be added is the same as the last one ⓬.

This works reasonably well in low-load situations, where there are suitable *quiescent* points at which no threads are in `pop()`. However, this is potentially a transient situation, which is why you need to test that the `threads_in_pop` count decrements to zero ❸ before doing the reclaim and why this test occurs *before* you delete the just-removed node ❼. Deleting a node is potentially a time-consuming operation, and you want the window in which other threads can modify the list to be as small as possible. The longer the time between when the thread first finds